

Part C — Deployment, Cloud & Integrations

Production-Grade Edition — revised to match how deployment is actually done at a company: IaC, reverse proxies, secrets management, blue-green rollouts, Kubernetes awareness, and real observability (metrics, tracing, dashboards) alongside the original Docker/CI-CD/hosting/integrations content.

Phases	7 – 9 (was 7–8; split into Deployment Core + Production Operations)
Topics	24 (was 12 — 12 new production topics added, 0 removed)
Est. Duration	10–12 weeks (was 6–7)
Difficulty	Intermediate → Advanced → Production
Suggested Pace	8–12 hrs/week · 2–3 topics/week
Projects	3–5 hands-on projects per topic, step-by-step
Doc Sources	3+ official/primary sources per topic

What Changed From v1

- Kept everything from v1 — Payment, Email, OAuth, File Storage, AI Integration, Docker, GitHub Actions, Cloud Compute, Cloud Storage, PaaS Hosting, Domains/HTTPS, Monitoring — with tightened formatting.
- Added 12 topics that were missing but are standard in any real production stack: Reverse Proxies & Nginx, Secrets & Environment Management, Infrastructure as Code (Terraform), Blue-Green Deployments & Rollbacks, Kubernetes (intro/awareness), CDN, Metrics & Prometheus, Grafana Dashboards, OpenTelemetry & Distributed Tracing, Queues & Async Processing — plus folded Logging into Error Tracking as one Observability arc.
- Split old Phase 8 into **Phase 8 — Deployment Core** (how the app ships and runs) and **Phase 9 — Production Operations** (how you keep it running and know when it breaks), because that is genuinely two different skill sets and cramming both into one phase was the main reason v1 read thin on production realism.
- Part D (Data Structures & Algorithms) now becomes Phase 10 in the master roadmap — no content lost, just renumbered.

Phase 7 — External APIs

Payment Integration · Email Integration · OAuth & Social Login · File Storage & Uploads · AI API Integration · *(unchanged from v1, formatting only)*

Phase 8 — Deployment Core

Docker & Containerization · **Reverse Proxies & Nginx** · **Secrets & Environment Management** · **Infrastructure as Code** · CI/CD with GitHub Actions · **Blue-Green Deployments & Rollbacks** · Cloud Compute Basics · **Kubernetes (Intro)**

Phase 9 — Production Operations & Reliability

Cloud Storage · **CDN** · PaaS Hosting · Domains & HTTPS · **Metrics & Prometheus** · **Grafana Dashboards** · **OpenTelemetry & Distributed Tracing** · Logging & Error Tracking · **Queues & Async Processing**

PHASE 7

PHASE 7 OF 10

External APIs — third-party integration: payment, email, auth providers, AI

Payment Integration

Accepting real (or test-mode) payments and handling the asynchronous confirmation that comes back after the charge.

Subtopics: Stripe/PayMongo checkout flow · payment intents · webhook confirmation · idempotency

1. Stripe/PayMongo Checkout

create a payment intent on the backend → confirm it client-side with the provider SDK → handle success/failure UI states → test with official test cards

2. Webhook for Payment Confirmation

build a webhook endpoint → verify the webhook signature → mark PAID only from the webhook, never the frontend → test with the provider's webhook CLI

3. Idempotent Webhook Handling

fire the same event twice → check by event ID before applying it → confirm order is marked PAID only once → log the duplicate, don't error on it

4. Failed Payment Handling

trigger a declined test card → confirm order stays PENDING → surface a clear error to the user → allow retry with a different card

5. Refund Flow

build an admin refund action → call the provider's refund API → update order status → confirm the refund webhook is also handled

Docs: Stripe Docs — stripe.com/docs | PayMongo Docs — developers.paymongo.com/docs | Stripe Webhooks Guide — stripe.com/docs/webhooks

Email Integration

Sending transactional emails triggered by real events in your app — confirmations and receipts, not marketing email.

Subtopics: SendGrid/Resend setup · transactional templates · triggering on backend events · deliverability basics

1. Order Confirmation Email

set up SendGrid or Resend → build an HTML template with the order summary → trigger from the order-placed event → confirm it lands in a real inbox

2. Password Reset Email

generate a time-limited reset token → email a link containing it → build the reset page it links to → confirm the token expires and can't be reused

3. Shipping Notification Email

trigger on status change to SHIPPED → include tracking info → test via the admin status update

4. Email Template Variables

build one base template → reuse for confirmation and shipping notice → pass different variables per use case

5. Failed Email Handling

simulate the provider being unreachable → confirm checkout still completes → log the failure for retry → add a retry-once mechanism

Docs: SendGrid Docs — docs.sendgrid.com | Resend Docs — resend.com/docs | Baeldung: Email with Spring — baeldung.com/spring-email

OAuth & Social Login

Letting users sign in with Google/GitHub instead of a new password, while still issuing your own JWT.

Subtopics: OAuth 2.0 authorization code flow · exchanging a code for a profile · linking to your user model · issuing your own JWT after OAuth

1. Google OAuth Login

register an OAuth app with Google → redirect to Google's consent screen → exchange the code for a profile → issue your own JWT on success

2. Account Linking

detect if the OAuth email matches an existing account → link instead of duplicating → test signing in via Google after registering by email

3. New User via OAuth

detect a brand-new OAuth email → auto-create a user record → redirect to profile completion if needed → confirm the JWT works immediately

4. OAuth Error Handling

simulate the user clicking deny → handle the error redirect gracefully → show a clear retry path

5. Second Provider: GitHub OAuth

add GitHub as a second provider → reuse the Google flow's logic → confirm both link to the same account by email

Docs: Google OAuth 2.0 — developers.google.com/identity/protocols/oauth2 | GitHub OAuth Apps — docs.github.com | Spring Security OAuth2 Client — docs.spring.io

File Storage & Uploads

Handling user-uploaded files by streaming them to cloud storage instead of the app server's own disk.

Subtopics: multipart form uploads · Cloudinary/S3 SDK basics · storing the URL not the file · validating type/size

1. Product Image Upload

build a multipart form in React → accept the file on a Spring Boot endpoint → upload to Cloudinary/S3 from the backend → store the returned URL on the Product entity

2. Drag-and-Drop Upload UI

replace the file input with a drop zone → show a preview and progress → handle a non-image file gracefully

3. File Validation

validate type on frontend and backend → cap file size → return a clear rejection error → test with an oversized file

4. Image Transformation on Upload

configure an automatic thumbnail transform → store full and thumbnail URLs → use thumbnail in list view, full image in detail view

5. Delete Orphaned Files

delete the product record → call the storage provider's delete API for its image → confirm no orphaned files remain

Docs: Cloudinary Docs — cloudinary.com/documentation | AWS S3 Guide — docs.aws.amazon.com/s3 | Baeldung: Multipart Upload — baeldung.com/spring-file-upload

AI API Integration

Calling an LLM API from your own backend to power a feature, not just chatting with one yourself.

Subtopics: prompting from server-side code · structured/JSON responses · rate limits & cost awareness · graceful fallback

1. AI Product Description Generator

send name + category to the Claude/OpenAI API → request a generated description → show it as an editable admin suggestion → handle the call failing gracefully

2. Structured JSON Response

prompt for strict JSON output → parse it on the backend → validate the shape before saving → handle malformed JSON without crashing

3. AI-Powered Search Assistant

accept a natural-language query → extract structured filters via the AI API → run those filters against real product search → return real results, not AI-generated ones

4. Rate Limit & Cost Guardrails

add a per-user rate limit → cache repeated identical requests briefly → log token usage per call → add a daily cap with a friendly fallback

5. Fallback When AI Is Unavailable

simulate the API being down/timing out → confirm the rest of the app still works → show a clear unavailable state instead of an error page

Docs: Anthropic API Docs — docs.claude.com | OpenAI API Docs — platform.openai.com/docs | Anthropic Prompt Engineering — docs.claude.com

Final project — Phase 7: PayMongo/Stripe real payment flow with webhook-driven PAID status · order-confirmation + shipping emails · Google OAuth alongside email/password · drag-and-drop product image upload on Cloudinary · AI-assisted search bar powered by the Claude API.

PHASE 8

PHASE 8 OF 10 — DEPLOYMENT CORE

How the app is packaged, secured, and shipped — Docker through Kubernetes awareness

Docker & Containerization

Packaging your app plus everything it needs to run into one portable unit that behaves identically everywhere.

Subtopics: images vs containers · Dockerfile · docker-compose for multi-service apps · volumes & networking

1. Dockerize Spring Boot

write a Dockerfile → build with docker build → run with docker run and hit the API → confirm parity with local Maven run

2. Dockerize the React Frontend

write a multi-stage build+nginx-serve Dockerfile → build and run the image → confirm the built frontend serves on the exposed port

3. docker-compose for the Full Stack

write a compose file with backend, frontend, PostgreSQL → network services by name → docker-compose up and confirm it all works together

4. Environment Variables in Docker

move DB credentials and JWT secret to env vars → pass via compose environment/.env → confirm the image has no hardcoded secrets

5. Volume for Persistent Data

add a named volume for Postgres data → stop/remove the container → start a new one against the same volume → confirm data survived

Docs: Docker Get Started — docs.docker.com/get-started | Dockerfile Reference — docs.docker.com/reference/dockerfile | Docker Compose — docs.docker.com/compose

Reverse Proxies & Nginx

The layer that actually receives internet traffic in front of your app — routing, TLS termination, and serving static files, instead of exposing Spring Boot or Node directly. This was missing in v1 and is present in essentially every real deployment.

Subtopics: reverse proxy vs load balancer · Nginx as a static file server · Nginx as a reverse proxy to your backend · path-based routing · gzip & caching headers

1. Nginx in Front of Spring Boot

run Nginx as a container alongside the backend → proxy_pass /api requests to the backend service → confirm the backend is no longer directly exposed

2. Serve the React Build via Nginx

copy the production build into an Nginx image → configure try_files for client-side routing → confirm deep links (e.g. /product/12) don't 404 on refresh

3. Single Entry Point for Frontend + API

route / to the frontend and /api to the backend from one Nginx config → confirm both work behind one port/domain → explain why this avoids CORS entirely

4. Rate Limiting at the Proxy

add an Nginx limit_req zone → hit an endpoint past the limit → confirm 429s are returned before the request reaches your app

Docs: Nginx Docs — nginx.org/en/docs | Nginx Reverse Proxy Guide — docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy | DigitalOcean: Nginx as Reverse Proxy — digitalocean.com/community/tutorials

Secrets & Environment Management

Keeping API keys, DB passwords, and JWT secrets out of source control and out of the container image itself — the single most common real-world security mistake junior developers make.

Subtopics: .env files & .gitignore discipline · per-environment config (dev/staging/prod) · platform secret stores · secret rotation basics

1. Environment-Per-Config Setup

create separate application-dev.yml / application-prod.yml profiles → select the profile via an env var at startup → confirm dev never touches prod credentials

2. Using a Platform Secret Store

move secrets out of .env into Railway/Render's or GitHub Actions' encrypted secret store → reference them as env vars at runtime/build time → confirm nothing sensitive is committed

3. Secret Scanning

run a secret scanner (e.g. gitleaks) against your repo history → confirm it flags anything you accidentally committed before → rotate any real credential that was ever exposed

4. Rotate a Credential

rotate the DB password or an API key → update it in the secret store only → redeploy and confirm the app picks up the new value with zero code changes

Docs: GitHub Actions: Encrypted Secrets — docs.github.com/actions/security-guides/encrypted-secrets | gitleaks — github.com/gitleaks/gitleaks | 12-Factor App: Config — 12factor.net/config

Infrastructure as Code

Defining your cloud resources (buckets, databases, compute) in version-controlled code instead of clicking through a console — so environments are reproducible and reviewable like any other code change.

Subtopics: declarative vs imperative provisioning · Terraform basics: providers, resources, state · plan vs apply · why console clicks don't scale

1. Terraform "Hello Cloud"

install Terraform → write a provider block for AWS/GCP → define one resource (e.g. an S3 bucket) → terraform plan then apply, confirm it exists in the console

2. Recreate an Earlier Manual Resource

pick a bucket or instance you created by hand earlier in this roadmap → define it in Terraform instead → import or recreate it → destroy the manual one

3. State File Awareness

inspect terraform.tfstate → explain why it must not be committed to a public repo → set up a remote state backend (S3 + DynamoDB lock, or Terraform Cloud)

4. Tear Down Cleanly

run terraform destroy on a test environment → confirm every resource it created is gone → compare this to manually hunting down console resources

Docs: Terraform Docs — developer.hashicorp.com/terraform/docs | Terraform AWS Provider — registry.terraform.io/providers/hashicorp/aws | HashiCorp Learn: Terraform Basics — developer.hashicorp.com/terraform/tutorials

CI/CD with GitHub Actions

Automatically testing and building your code on every push, so broken code never reaches production silently.

Subtopics: workflow YAML basics · running tests on push · building & pushing a Docker image · deploying on merge to main

1. First GitHub Actions Workflow

add .github/workflows/ci.yml → trigger on push and pull_request → run mvn test / npm test → confirm red X on failure, green check on pass

2. Multi-Job Pipeline

split into backend-test and frontend-test jobs → run them in parallel → confirm both must pass

3. Build & Push a Docker Image

build the image on merge to main → push to Docker Hub/GHCR → tag with the commit SHA → confirm it appears in the registry

4. Require CI to Pass Before Merge

enable branch protection on main → require the workflow to pass → confirm a failing PR literally cannot merge

5. Auto-Deploy on Merge

add a deploy job on push to main only → call the hosting provider's deploy hook/CLI → confirm a merge results in a live update automatically

Docs: GitHub Actions Docs — docs.github.com/en/actions | Building/Testing Java with Maven — docs.github.com/en/actions | Workflow Syntax Reference — docs.github.com/en/actions

NEW · MODERN PRODUCTION
TOPIC

Blue-Green Deployments & Rollbacks

Shipping a new version with zero downtime, and — just as important — having a fast, boring way to undo it when something goes wrong in production. v1 never addressed what happens after a bad deploy; this does.

Subtopics: blue-green vs rolling vs canary · health checks before traffic switch · one-command rollback · database migration safety during deploys

1. Blue-Green on a PaaS

deploy a new version alongside the running one → run a health check against it before switching traffic → flip traffic to the new version → keep the old one warm for instant rollback

2. Practice a Rollback

deploy a deliberately broken version → detect the failure via health check or monitoring → roll back to the previous known-good deploy → time how long it took

3. Backward-Compatible DB Migrations

add a nullable column instead of a breaking schema change → deploy code that works with both old and new schema → backfill data → only then make the column required in a later deploy

4. Canary Awareness

read how canary releases differ from blue-green → explain when you'd choose canary over blue-green for ShopFlow → no build required — this one's a reasoning exercise

Docs: Martin Fowler: BlueGreenDeployment — martinfowler.com/bliki/BlueGreenDeployment.html | Render: Zero-Downtime Deploys — render.com/docs | Google SRE Book: Release Engineering — sre.google/sre-book

Cloud Compute Basics

Where your backend actually runs once it's not localhost — awareness of the major providers, hands-on with at least one.

Subtopics: AWS EC2/Elastic Beanstalk · Google Cloud Run/App Engine awareness · Azure App Service awareness · compute vs PaaS

1. Launch a Basic EC2 Instance

launch a free-tier instance → SSH into it → install Java and run your Spring Boot JAR manually → hit the API over its public IP

2. Deploy via Elastic Beanstalk

package the app for EB → deploy via EB CLI/console → confirm it runs behind a managed load balancer → compare effort to raw EC2

3. Google Cloud Run Deployment

build your Docker image → deploy it to Cloud Run → confirm it scales to zero when idle → compare cost/complexity to EC2

4. Compute Provider Comparison

note setup time, pricing model, ease of use for each → decide your default for a personal vs a company project

5. Shut Everything Down

terminate every resource you created → confirm nothing is running in the billing dashboard → make this a habit going forward

Docs: AWS EC2 Guide — docs.aws.amazon.com/ec2 | AWS Elastic Beanstalk Guide — docs.aws.amazon.com/elasticbeanstalk | Google Cloud Run Docs — cloud.google.com/run/docs

NEW · MODERN PRODUCTION
TOPIC

Kubernetes (Intro Only)

Awareness-level only — enough to read a job posting or a company's infra diagram and know what's being described. Not a substitute for a dedicated Kubernetes track, and ShopFlow does not need to actually run on it.

Subtopics: pods, deployments, services · why K8s exists beyond docker-compose · kubectl basics · when a team actually needs it vs when PaaS is enough

1. Local Cluster with minikube/kind

spin up a local cluster → kubectl get nodes to confirm it's running

2. Deploy One Container as a Pod

write a minimal Deployment YAML for your backend image → kubectl apply it → kubectl get pods and confirm it's Running

3. Expose It with a Service

add a Service YAML → port-forward or use a NodePort → hit the API through the cluster

4. Scale and Observe

kubectl scale to 3 replicas → kubectl get pods and watch them come up → kill one pod manually and watch Kubernetes replace it

5. Write Down the "Why"

in 3-4 sentences, explain what problem Kubernetes solves that docker-compose doesn't → note that most solo/small-team projects should stay on PaaS until they outgrow it

Docs: Kubernetes Docs — kubernetes.io/docs | kubectl Quick Reference — kubernetes.io/docs/reference/kubectl | minikube — minikube.sigs.k8s.io

Final project — Phase 8: ShopFlow fully containerized (Docker + Compose) and served through Nginx · all secrets in a platform secret store, none in git · core infra (one bucket + one compute resource) defined in Terraform · GitHub Actions pipeline builds, tests, and deploys on merge · a practiced rollback with a documented time-to-recover · one component running as a Kubernetes Deployment locally, purely for the exercise.

PHASE 9

PHASE 9 OF 10 — PRODUCTION OPERATIONS & RELIABILITY

Storage, CDN, domains, and knowing what's happening inside a live system

Cloud Storage

Storing files outside your application server, in a service built to hold them reliably at any scale.

Subtopics: AWS S3 buckets & objects · Google Cloud Storage basics · Cloudinary (media-focused) · public vs private access & signed URLs

1. Create and Use an S3 Bucket

create a bucket via console → upload manually to confirm it works → upload programmatically from Spring Boot via the AWS SDK → retrieve it via its URL

2. Public vs Private Bucket Access

set the bucket private by default → generate a pre-signed URL with a short expiry → confirm the file isn't accessible without it, is with it, until it expires

3. Google Cloud Storage Equivalent

create a GCS bucket → upload/retrieve from your backend → compare the SDK to S3

4. Cloudinary for Media-Specific Needs

revisit your Phase 7 upload integration → add an automatic resize/format transform → decide S3 vs Cloudinary for product images

5. Storage Cost & Lifecycle Awareness

read the provider's pricing page → set a lifecycle rule to auto-delete after N days in a test bucket → delete all test buckets when done

Docs: AWS S3 Guide — docs.aws.amazon.com/s3 | Google Cloud Storage Docs — cloud.google.com/storage/docs | Cloudinary Docs — cloudinary.com/documentation

CDN

Serving static assets (JS/CSS bundles, product images) from edge locations close to the user instead of your origin server — the difference between a site that feels fast everywhere and one that's only fast near your host's region.

Subtopics: what a CDN actually caches · origin vs edge · cache invalidation on deploy · CDN in front of S3/Cloudinary vs built into your PaaS

1. Confirm Your Frontend Is Already Behind One

check response headers on your Vercel/Netlify-deployed frontend → identify the CDN provider from the headers → note which assets are cached vs not

2. CDN in Front of S3

put CloudFront (or equivalent) in front of your S3 bucket → confirm images load faster on repeat requests → confirm the bucket itself can stay private

3. Cache Invalidation on Deploy

change a static asset → redeploy → confirm the CDN serves the new version, not a stale cached one → invalidate manually if it doesn't

Docs: MDN: CDN Overview — developer.mozilla.org/en-US/docs/Glossary/CDN | AWS CloudFront Docs — docs.aws.amazon.com/cloudfront | Cloudflare Learning: CDN — cloudflare.com/learning/cdn

PaaS Hosting (Vercel, Netlify, Railway, Render)

The fastest path from GitHub repo to live URL — platforms that handle server management for you.

Subtopics: frontend hosting (Vercel/Netlify) · backend hosting (Railway/Render) · auto-deploy on push · preview URLs per PR

1. Deploy the Frontend

connect the repo to Vercel/Netlify → configure the build command/output dir → confirm auto-deploy on push → open a PR and confirm a preview URL

2. Deploy the Backend

connect the backend repo/Docker image → set env vars in the dashboard → confirm the API is reachable → connect a managed PostgreSQL instance

3. Environment-Based API URLs

add a build-time API base URL variable → confirm prod frontend calls prod backend → confirm local dev still points at localhost

4. Preview Environments for PRs

open a PR with a visible change → confirm a unique preview URL → merge and confirm it promotes to production

5. Compare PaaS vs Cloud Compute

list what the PaaS handled automatically → list what you did manually on EC2 → decide which you'd pick for ShopFlow, and why

Docs: Vercel Docs — vercel.com/docs | Netlify Docs — docs.netlify.com | Railway Docs — docs.railway.com

Domains & HTTPS

Pointing a real domain at your deployed app, and making sure every connection to it is encrypted.

Subtopics: DNS records (A/CNAME) · connecting a custom domain · HTTPS & SSL certificates · why HTTPS is non-negotiable

1. Point a Domain at Vercel/Netlify

add the CNAME/A record your host specifies → wait for DNS propagation → confirm the domain loads the frontend

2. Subdomain for the API

add a CNAME for api.yourdomain.com → update the frontend's API base URL → confirm CORS allows the frontend domain

3. Confirm Automatic HTTPS

confirm a valid certificate in the browser → inspect who issued it and how it auto-renews

4. DNS Record Types Drill

look up your A, CNAME, and MX records → explain what each does → explain why an A and CNAME can't both target the root

5. Force HTTPS Redirect

load the site over plain http:// → confirm it force-redirects → configure the redirect explicitly if it doesn't

Docs: MDN: What Is a Domain — developer.mozilla.org/en-US/docs/Learn_web_development | Cloudflare: DNS Records — cloudflare.com/learning/dns | MDN: What Is HTTPS — developer.mozilla.org/en-US/docs/Glossary/HTTPS

Metrics & Prometheus

Numbers about your system over time — request rate, latency, error rate, resource usage — collected and stored so you can graph them, not just glimpsed once in a log line.

Subtopics: counters, gauges, histograms · the four golden signals · Micrometer in Spring Boot · Prometheus scraping model

1. Expose Metrics from Spring Boot

add Micrometer + the Prometheus registry → expose /actuator/prometheus → confirm request-count and latency metrics appear

2. Run Prometheus and Scrape Them

run Prometheus via Docker → point it at your app's metrics endpoint → confirm it's scraping on the target's status page

3. Golden Signals Query

write a Prometheus query for request rate → write one for p95 latency → write one for error rate → explain what each tells you that logs alone don't

4. Custom Business Metric

add a custom counter for e.g. orders_placed_total → trigger a few test orders → confirm the counter increments in Prometheus

Docs: Prometheus Docs — prometheus.io/docs | Micrometer Docs — micrometer.io/docs | Spring Boot Actuator — docs.spring.io/spring-boot/reference/actuator

Grafana Dashboards

Turning Prometheus's raw numbers into a dashboard a human can actually glance at during an incident.

Subtopics: connecting a data source · building panels · dashboard variables · alert rules from a panel

1. First Dashboard

run Grafana via Docker → add Prometheus as a data source → build a panel for request rate and one for p95 latency

2. Import a Community Dashboard

import a JVM/Spring Boot community dashboard → confirm it populates from your app's metrics → trim it to what's actually useful for ShopFlow

3. Alert on a Threshold

create an alert rule for error rate > X% → trigger it by generating errors → confirm the alert fires in Grafana

Docs: Grafana Docs — grafana.com/docs | Grafana + Prometheus Getting Started — grafana.com/docs/grafana/latest/getting-started | Awesome Prometheus Dashboards (community) — github.com

OpenTelemetry & Distributed Tracing

Following a single request as it crosses your frontend, backend, database, and any third-party API call — essential the moment your system is more than one service, and increasingly the industry-standard instrumentation layer.

Subtopics: spans & traces · context propagation across services · OpenTelemetry SDK basics · exporting to a tracing backend

1. Instrument Spring Boot with OpenTelemetry

add the OpenTelemetry Java agent or SDK → auto-instrument HTTP requests and DB calls → confirm spans are generated for a single request

2. View a Trace End-to-End

export traces to Jaeger or a hosted backend → make a request that hits the DB and an external API → view the full trace as one waterfall, not scattered logs

3. Correlate a Trace with an Error

trigger a deliberate error deep in the call chain → find its trace → confirm you can see exactly which span failed and why

Docs: OpenTelemetry Docs — opentelemetry.io/docs | OTel Java Instrumentation — github.com/open-telemetry/opentelemetry-java-instrumentation | Jaeger Tracing — jaegertracing.io

Logging & Error Tracking

Finding out about production problems from a dashboard, not from an angry user — combines v1's Monitoring and Error Tracking topics into one observability arc alongside metrics and tracing above.

Subtopics: uptime monitoring · error tracking (Sentry) · structured production logging · alerting

1. Uptime Monitor Setup

sign up for a free uptime monitor → point it at your frontend and backend URLs → configure a downtime alert → test by stopping the backend briefly

2. Sentry Error Tracking

add Sentry to frontend and backend → trigger a deliberate error in each → confirm it appears with a full stack trace → add user context

3. Structured Logging Review

confirm SLF4J logs appear in your host's log viewer → search/filter for a specific request → confirm no sensitive data appears in production logs

4. Basic Alerting Rule

configure an alert for an error-rate spike, not just downtime → trigger several errors in a short window → confirm the alert fires

5. Incident Postmortem Practice

pick a bug you fixed earlier in the roadmap → write a short postmortem: what happened, why, what changed → note one monitoring improvement that would have caught it sooner

Docs: Sentry Docs — docs.sentry.io | UptimeRobot — uptimerobot.com | Grafana: Getting Started — grafana.com/docs/grafana/latest/getting-started

NEW · MODERN PRODUCTION
TOPIC

Queues & Async Processing

Moving slow or non-critical work (emails, image processing, AI calls) off the request/response cycle so a user's checkout doesn't wait on a third-party API that might be slow.

Subtopics: message queue basics (producer/consumer) · RabbitMQ or SQS · retry & dead-letter queues · when to queue vs call synchronously

1. Move Order Emails to a Queue

publish an OrderPlaced event to a queue on checkout → build a worker that consumes it and sends the email → confirm checkout responds instantly, email arrives moments later

2. Retry with a Dead-Letter Queue

make the email worker fail intentionally a few times → confirm it retries with backoff → confirm it lands in a dead-letter queue after max retries instead of looping forever

3. Queue the AI Description Generator

move the Phase 7 AI product-description call off the request path → return immediately with a "generating..." state → update the product once the worker finishes

4. Monitor Queue Depth

expose queue depth as a metric → watch it spike under a burst of test orders → confirm it drains back to zero

Docs: RabbitMQ Docs — rabbitmq.com/docs | AWS SQS Developer Guide — docs.aws.amazon.com/sqs | Spring AMQP Reference — docs.spring.io/spring-amqp/reference

Final project — Phase 9 (and Part C overall): Backend containerized and behind Nginx, deployed via Docker to Railway/Render, connected to hosted PostgreSQL · frontend on Vercel/Netlify behind a CDN with environment-based API URLs · core infra defined in Terraform, all secrets in a platform secret store · GitHub Actions running tests on every push, deploying on merge, with a rehearsed rollback path · product images in S3 or Cloudinary · custom domain with HTTPS, api. subdomain for the backend · Prometheus + Grafana dashboard showing the four golden signals · OpenTelemetry tracing across a real request · Sentry catching real errors, uptime monitoring alerting on downtime · order emails and AI calls running through a queue instead of blocking checkout.

This is the version of "deployed" that matches what a production engineering team actually maintains — not just a live URL, but one that's observable, rollback-able, and reproducible from code. Part D (Data Structures & Algorithms) becomes Phase 10.

Part C v2 — Progress Tracker

Check off each topic once concepts are read, projects are built, and the result is folded into ShopFlow.

PHASE 7 — External APIs

- Payment Integration
- Email Integration
- OAuth & Social Login
- File Storage & Uploads
- AI API Integration

PHASE 8 — Deployment Core

- Docker & Containerization

- Reverse Proxies & Nginx (new)
- Secrets & Environment Management (new)
- Infrastructure as Code (new)
- CI/CD with GitHub Actions
- Blue-Green Deployments & Rollbacks (new)
- Cloud Compute Basics
- Kubernetes — Intro Only (new)

PHASE 9 — Production Operations & Reliability

- Cloud Storage
- CDN (new)
- PaaS Hosting
- Domains & HTTPS
- Metrics & Prometheus (new)
- Grafana Dashboards (new)
- OpenTelemetry & Distributed Tracing (new)
- Logging & Error Tracking
- Queues & Async Processing (new)